

Red Hat Data Grid 8.6

Using Data Grid with Spring

Add Data Grid to Spring applications

Last Updated: 2026-06-03

Red Hat Data Grid 8.6 Using Data Grid with Spring

Add Data Grid to Spring applications

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Add Data Grid caching capabilities to Spring-based applications.

Table of Contents

RED HAT DATA GRID 3

DATA GRID DOCUMENTATION 4

DATA GRID DOWNLOADS 5

MAKING OPEN SOURCE MORE INCLUSIVE 6

CHAPTER 1. USING DATA GRID AS A SPRING CACHE PROVIDER 7
1.1. UPGRADING SPRING INTEGRATION IN DATA GRID 7
1.1.1. From Spring 6 to Spring 7 7
1.1.2. From Spring Boot 3.x to Spring Boot 4.x 7
1.2. SETTING UP SPRING CACHING WITH DATA GRID 7
Spring Cache dependencies 8
1.3. USING DATA GRID AS A SPRING CACHE PROVIDER 9
1.4. SPRING CACHE ANNOTATIONS 10
@Cacheable 10
@CacheEvict 10
1.5. CONFIGURING TIMEOUTS FOR CACHE OPERATIONS 11

CHAPTER 2. EXTERNALIZING SESSIONS WITH SPRING SESSION 13
2.1. EXTERNALIZING SESSIONS WITH SPRING SESSION 13

RED HAT DATA GRID

Data Grid is a high-performance, distributed in-memory data store.

Schemaless data structure

Flexibility to store different objects as key-value pairs.

Grid-based data storage

Designed to distribute and replicate data across clusters.

Elastic scaling

Dynamically adjust the number of nodes to meet demand without service disruption.

Data interoperability

Store, retrieve, and query data in the grid from different endpoints.

DATA GRID DOCUMENTATION

Documentation for Data Grid is available on the Red Hat customer portal.

- Data Grid 8.6 Documentation
- Data Grid 8.6 Component Details
- Supported Configurations for Data Grid 8.6
- Data Grid 8 Feature Support
- Data Grid Deprecated Features and Functionality

DATA GRID DOWNLOADS

Access the Data Grid Software Downloads on the Red Hat customer portal.



NOTE

You must have a Red Hat account to access and download Data Grid software.

MAKING OPEN SOURCE MORE INCLUSIVE

Red Hat is committed to replacing problematic language in our code, documentation, and web properties. We are beginning with these four terms: master, slave, blacklist, and whitelist. Because of the enormity of this endeavor, these changes will be implemented gradually over several upcoming releases. For more details, see our CTO Chris Wright's message.

CHAPTER 1. USING DATA GRID AS A SPRING CACHE PROVIDER

Add Data Grid dependencies to your application and use Spring Cache annotations to store data in embedded or remote caches.

1.1. UPGRADING SPRING INTEGRATION IN DATA GRID

1.1.1. From Spring 6 to Spring 7

Data Grid supports both Spring 6 and Spring 7. To migrate, simply update your Maven dependencies.

- **infinispan-spring6-embedded** by **infinispan-spring7-embedded**
- **infinispan-spring6-remote** by **infinispan-spring7-remote**

Backward compatibility is maintained, and no functionality should be affected.

1.1.2. From Spring Boot 3.x to Spring Boot 4.x

Data Grid supports both Spring Boot 3.x and Spring Boot 4.x. To migrate, first update your Maven dependencies. * **infinispan-spring-boot3-starter-embedded** by **infinispan-spring-boot4-starter-embedded** * **infinispan-spring-boot3-starter-remote** by **infinispan-spring-boot4-starter-remote**

In Embedded mode, the **InfinispanConfigurationCustomizer** bean has been deprecated in Spring Boot 3.x and removed in Spring Boot 4 integration. This component was used to customize the default cache, which was automatically created. Going forward, use **InfinispanCacheConfigurer** and explicitly define a cache named **default** if needed.

1.2. SETTING UP SPRING CACHING WITH DATA GRID

Add the Data Grid dependencies to your Spring application project. If you use remote caches in a Data Grid Server deployment, you should also configure your Hot Rod client properties.



IMPORTANT

Data Grid supports Spring version version 6 and version 7.

Procedure

1. Add Data Grid and the Spring integration module to your **pom.xml**.
   - Spring 6 Remote caches: **infinispan-spring6-remote**
   - Spring 7 Remote caches: **infinispan-spring7-remote**
   - Spring 6 Embedded caches: **infinispan-spring6-embedded**
   - Spring 7 Embedded caches: **infinispan-spring7-embedded**

TIP

Spring Boot users can add the following artifacts instead of the **infinispan-spring6-embedded** or **infinispan-spring7-embedded**:

- For Spring Boot 3.x add **infinispan-spring-boot3-starter-embedded**
- For Spring Boot 4.x add **infinispan-spring-boot4-starter-embedded**

2. Configure your Hot Rod client to connect to your Data Grid Server deployment in the **hotrod-client.properties** file.

```
infinispan.client.hotrod.server_list = 127.0.0.1:11222
infinispan.client.hotrod.auth_username=admin
infinispan.client.hotrod.auth_password=changeme
```

Spring Cache dependencies

Remote caches Spring 6

```
<dependencies>
   <dependency>
      <groupId>org.infinispan</groupId>
      <artifactId>infinispan-spring6-remote</artifactId>
   </dependency>
   <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>${version.spring}</version>
   </dependency>
</dependencies>
```

Remote caches Spring 7

```
<dependencies>
   <dependency>
      <groupId>org.infinispan</groupId>
      <artifactId>infinispan-spring7-remote</artifactId>
   </dependency>
   <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>${version.spring}</version>
   </dependency>
</dependencies>
```

Embedded caches Spring 6

```
<dependencies>
   <dependency>
      <groupId>org.infinispan</groupId>
      <artifactId>infinispan-spring6-embedded</artifactId>
```

```
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>${version.spring}</version>
    </dependency>
</dependencies>
```

Embedded caches Spring 7

```
<dependencies>
    <dependency>
      <groupId>org.infinispan</groupId>
      <artifactId>infinispan-spring7-embedded</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>${version.spring}</version>
    </dependency>
</dependencies>
```

Additional resources

- Configuring Hot Rod Client connections

1.3. USING DATA GRID AS A SPRING CACHE PROVIDER

Add the **@EnableCaching** annotation to one of your configuration classes and then add the **@Cacheable** and **@CacheEvict** annotations to use remote or embedded caches.

Prerequisites

- Add the Data Grid dependencies to your application project.
- Create the required remote caches and configure Hot Rod client properties if you use a Data Grid Server deployment.

Procedure

1. Enable cache annotations in your application context in one of the following ways:

   Declarative

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:cache="http://www.springframework.org/schema/cache"
    xmlns:p="http://www.springframework.org/schema/p"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/cache
http://www.springframework.org/schema/cache/spring-cache.xsd">
```

```
        <cache:annotation-driven />

</beans>
```

Programmatic

```
@EnableCaching @Configuration
public class Config {
}
```

2. Annotate methods with **@Cacheable** to cache return values.

   TIP

   To reference entries in the cache directly, you must include the **key** attribute.

3. Annotate methods with **@CacheEvict** to remove old entries from the cache.

Additional resources

- Spring Framework - Default Key Generation

1.4. SPRING CACHE ANNOTATIONS

The **@Cacheable** and **@CacheEvict** annotations add cache capabilities to methods.

@Cacheable

Stores return values in a cache.

@CacheEvict

Controls cache size by removing old entries.

@Cacheable

Taking **Book** objects as an example, if you want to cache each instance after loading it from a database with a method such as **BookDao#findBook(Integer bookId)**, you could add the **@Cacheable** annotation as follows:

```
@Transactional
@Cacheable(value = "books", key = "#bookId")
public Book findBook(Integer bookId) {...}
```

With the preceding example, when **findBook(Integer bookId)** returns a **Book** instance it gets stored in the cache named **books**.

@CacheEvict

With the **@CacheEvict** annotation, you can specify if you want to evict the entire **books** cache or only the entries that match a specific #bookId.

Entire cache eviction

Annotate the **deleteAllBookEntries()** method with **@CacheEvict** and add the **allEntries** parameter as follows:

```
@Transactional
@CacheEvict (value="books", key = "#bookId", allEntries = true)
public void deleteAllBookEntries() {...}
```

Entry based eviction

Annotate the **deleteBook(Integer bookId)** method with **@CacheEvict** and specify the key associated to the entry as follows:

```
@Transactional
@CacheEvict (value="books", key = "#bookId")
public void deleteBook(Integer bookId) {...}
```

1.5. CONFIGURING TIMEOUTS FOR CACHE OPERATIONS

The Data Grid Spring Cache provider defaults to blocking behaviour when performing read and write operations. Cache operations are synchronous and do not time out.

If necessary you can configure a maximum time to wait for operations to complete before they time out.

Procedure

- Configure the following timeout properties in the context XML for your application on either **SpringEmbeddedCacheManagerFactoryBean** or **SpringRemoteCacheManagerFactoryBean**.
  For remote caches, you can also add these properties to the **hotrod-client.properties** file.

Property	Description
infinispan.spring.operation.read.timeout	Specifies the time, in milliseconds, to wait for read operations to complete. The default is **0** which means unlimited wait time.
infinispan.spring.operation.write.timeout	Specifies the time, in milliseconds, to wait for write operations to complete. The default is **0** which means unlimited wait time.

The following example shows the timeout properties in the context XML for **SpringRemoteCacheManagerFactoryBean**:

```
<bean id="springRemoteCacheManagerConfiguredUsingConfigurationProperties"
      class="org.infinispan.spring.remote.provider.SpringRemoteCacheManagerFactoryBean">
   <property name="configurationProperties">
      <props>
         <prop key="infinispan.spring.operation.read.timeout">500</prop>
         <prop key="infinispan.spring.operation.write.timeout">700</prop>
      </props>
   </property>
</bean>
```

CHAPTER 2. EXTERNALIZING SESSIONS WITH SPRING SESSION

Store session data for Spring applications in Data Grid caches and independently of the container.

2.1. EXTERNALIZING SESSIONS WITH SPRING SESSION

Use the Spring Session API to externalize session data to Data Grid.

Procedure

1. Add dependencies to your **pom.xml**.
   - Embedded caches: **infinispan-spring6-embedded** or **infinispan-spring7-embedded**
   - Remote caches: **infinispan-spring6-remote** or **infinispan-spring7-remote**
     The following example is for remote caches in Spring 6:

```
<dependencies>
  <dependency>
    <groupId>org.infinispan</groupId>
    <artifactId>infinispan-core</artifactId>
  </dependency>
  <dependency>
    <groupId>org.infinispan</groupId>
    <artifactId>infinispan-spring6-remote</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>${version.spring}</version>
  </dependency>
  <dependency>
    <groupId>org.springframework.session</groupId>
    <artifactId>spring-session-core</artifactId>
    <version>${version.spring}</version>
  </dependency>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-web</artifactId>
    <version>${version.spring}</version>
  </dependency>
</dependencies>
```

2. Specify the appropriate **FactoryBean** to expose a **CacheManager** instance.
   - Embedded caches: **SpringEmbeddedCacheManagerFactoryBean**
   - Remote caches: **SpringRemoteCacheManagerFactoryBean**
3. Enable Spring Session with the appropriate annotation.
   - Embedded caches: **@EnableInfinispanEmbeddedHttpSession**

- Remote caches: **@EnableInfinispanRemoteHttpSession**
  These annotations have optional parameters:
  - **maxInactiveIntervalInSeconds** sets session expiration time in seconds. The default is **1800**.
  - **cacheName** specifies the name of the cache that stores sessions. The default is **sessions**.

The following example shows a complete, annotation-based configuration:

```
@EnableInfinispanEmbeddedHttpSession
@Configuration
public class Config {

  @Bean
  public SpringEmbeddedCacheManagerFactoryBean springCacheManager() {
    return new SpringEmbeddedCacheManagerFactoryBean();
  }

  //An optional configuration bean responsible for replacing the default
  //cookie that obtains configuration.
  //For more information refer to the Spring Session documentation.
  @Bean
  public HttpSessionIdResolver httpSessionIdResolver() {
     return HeaderHttpSessionIdResolver.xAuthToken();
  }
}
```